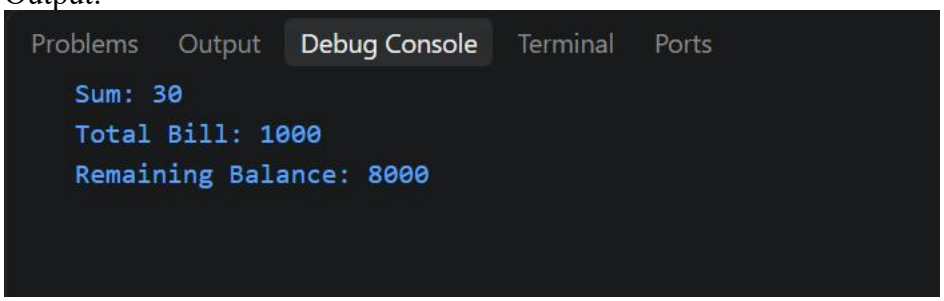
Question 1 - Callback-Based Calculator and Bill
Q1_Callbacks.js

```
function calculate(num1, num2, callback) {
    callback(num1, num2);
}
function addNumbers(a, b) {
    console.log("Sum:", a + b);
}
calculate(10, 20, addNumbers);
// Calculate the total bill
function calculateBill(price1, price2, price3, callback) {
    callback(price1 + price2 + price3);
}
function displayBill(total) {
    console.log("Total Bill:", total);
}
calculateBill(200, 300, 500, displayBill);
// Withdraw money after 2 seconds
function withdrawMoney(amount, callback) {
    let balance = 10000;
    setTimeout(function () {
        balance = balance - amount;
        callback(balance);
    }, 2000);
}
withdrawMoney(2000, function (remainingBalance) {
    console.log("Remaining Balance:", remainingBalance);
});
```

Output:

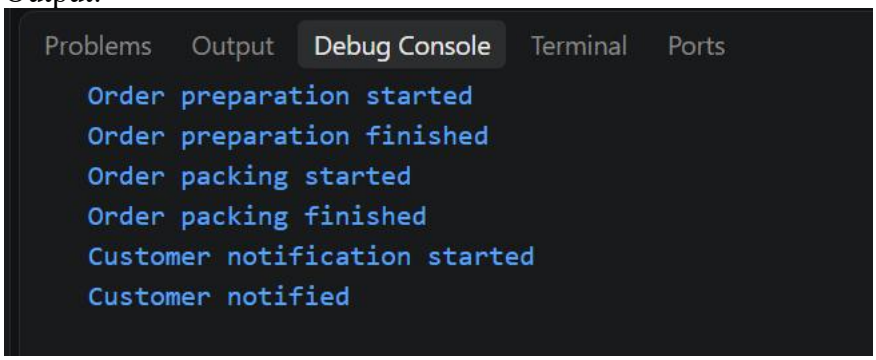


Question 2 - Order Processing Sequence
Q2_OrderProcessing.js
Version 1: Nested callbacks (Callback Hell)

```
function prepareOrder(callback) {
    console.log("Order preparation started");
    setTimeout(function () {
        console.log("Order preparation finished");
        callback();
    }, 1000);
}
function packOrder(callback) {
    console.log("Order packing started");
    setTimeout(function () {
        console.log("Order packing finished");
```

```
        callback();
    }, 1000);
}
function notifyCustomer() {
    console.log("Customer notification started");
    setTimeout(function () {
        console.log("Customer notified");
    }, 1000);
}
prepareOrder(function () {
    packOrder(function () {
        notifyCustomer();
    });
});
```

Output:



Version 2: Promise chaining

```
function prepareOrderPromise() {
    console.log("Order preparation started");

    return new Promise(function (resolve) {
        setTimeout(function () {
            console.log("Order preparation finished");
            resolve();
        }, 1000);
    });
}
function packOrderPromise() {
    console.log("Order packing started");
    return new Promise(function (resolve) {
        setTimeout(function () {
            console.log("Order packing finished");
            resolve();
        }, 1000);
    });
}
function notifyCustomerPromise() {
    console.log("Customer notification started");
    return new Promise(function (resolve) {
        setTimeout(function () {
            console.log("Customer notified");
            resolve();
        }, 1000);
    });
```

```
}
prepareOrderPromise()
    .then(function () {
        return packOrderPromise();
    })
    .then(function () {
        return notifyCustomerPromise();
    })
    .catch(function (error) {
        console.log("Error:", error);
    });
```

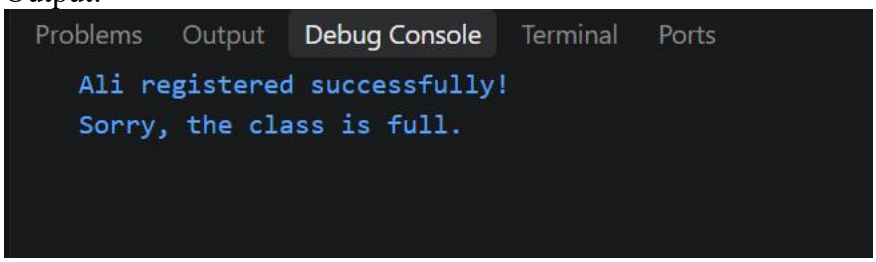
Output:

Question 3 - Event Registration Promise:

Q3_Promises.js

```
function registerStudent(studentName, seatAvailable) {
    return new Promise(function (resolve, reject) {
        if (seatAvailable) {
            resolve(studentName + " registered successfully!");
        } else {
            reject(new Error("Sorry, the class is full."));
        }
    });
}
// Test 1: Seat available
registerStudent("Ali", true)
    .then(function (message) {
        console.log(message);
    })
    .catch(function (error) {
        console.log(error.message);
    });
// Test 2: No seat available
registerStudent("Sara", false)
    .then(function (message) {
        console.log(message);
    })
    .catch(function (error) {
        console.log(error.message);
    });
```

Output:



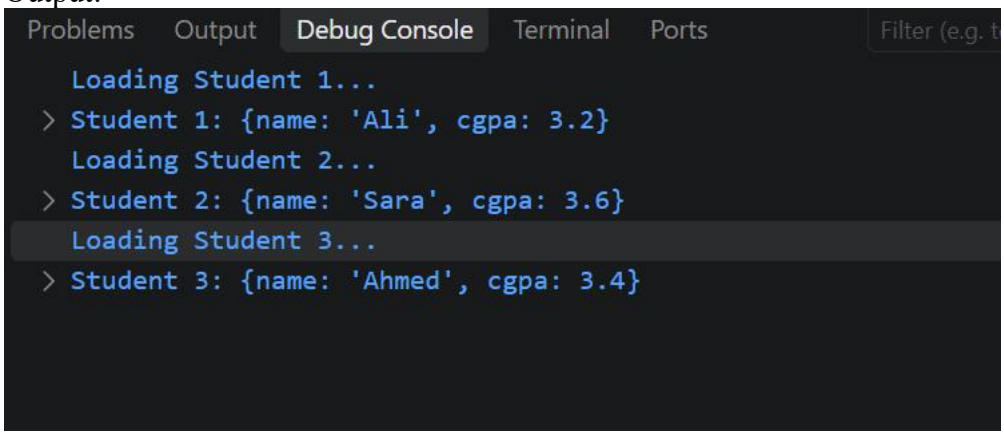
Question 4 - Load Student Records in Sequence

Q4_AsyncAwait.js

```
function getStudentData(id) {
    return new Promise(function (resolve, reject) {
```

```
        console.log("Loading Student " + id + "...");
        setTimeout(function () {
            const students = {
                1: { name: "Ali", cgpa: 3.2 },
                2: { name: "Sara", cgpa: 3.6 },
                3: { name: "Ahmed", cgpa: 3.4 }
            };
            if (students[id]) {
                resolve(students[id]);
            } else {
                reject(new Error("Student not found."));
            }
        }, 2000);
    });
}
async function loadStudents() {
    try {
        const student1 = await getStudentData(1);
        console.log("Student 1:", student1);
        const student2 = await getStudentData(2);
        console.log("Student 2:", student2);
        const student3 = await getStudentData(3);
        console.log("Student 3:", student3);
    } catch (error) {
        console.log("Error:", error.message);
    }
}
loadStudents();
```

Output:



Question 6 - Mini Project: Support Ticket Status Checker

Q6_TicketChecker.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Support Ticket Checker</title>
</head>
<body>
    <h2>Support Ticket Status Checker</h2>
    <label for="ticketId">Enter Ticket ID:</label>
    <input type="text" id="ticketId" placeholder="e.g. T101">
```

```
    <button id="checkButton">Check Status</button>
    <p id="result"></p>
    <script src="Q6_TicketChecker.js"></script>
</body>
</html>
```

Q6_TicketChecker.js

```
const tickets = [
    {
        ticketId: "T101",
        customerName: "Ali",
        status: "Open"
    },
    {
        ticketId: "T102",
        customerName: "Sara",
        status: "In Progress"
    },
    {
        ticketId: "T103",
        customerName: "Ahmed",
        status: "Resolved"
    }
];
function getTicket(ticketId) {
    return new Promise(function (resolve, reject) {
        setTimeout(function () {
            const ticket = tickets.find(function (item) {
                return item.ticketId === ticketId;
            });
            if (ticket) {
                resolve(ticket);
            } else {
                reject(new Error("Ticket ID not found. Please check the
ID."));
            }
        }, 2000);
    });
}
async function checkTicket() {
    const ticketId = document.getElementById("ticketId").value.trim();
    const result = document.getElementById("result");
    if (ticketId === "") {
        result.textContent = "Please enter a ticket ID.";
        return;
    }
    result.textContent = "Loading ticket details...";
    try {
        const ticket = await getTicket(ticketId);
        result.textContent =
            "Ticket ID: " + ticket.ticketId +
            " | Customer: " + ticket.customerName +
            " | Status: " + ticket.status;
    } catch (error) {
        result.textContent = error.message;
```

```
    }
}
document.getElementById("checkButton").addEventListener(
    "click",
    checkTicket
);
```

Output:



Support Ticket Status Checker

Enter Ticket ID: T101 Check Status

Loading ticket details...

Support Ticket Status Checker

Enter Ticket ID: T101 Check Status

Ticket ID: T101 | Customer: Ali | Status: Open

Support Ticket Status Checker

Enter Ticket ID: T102 Check Status

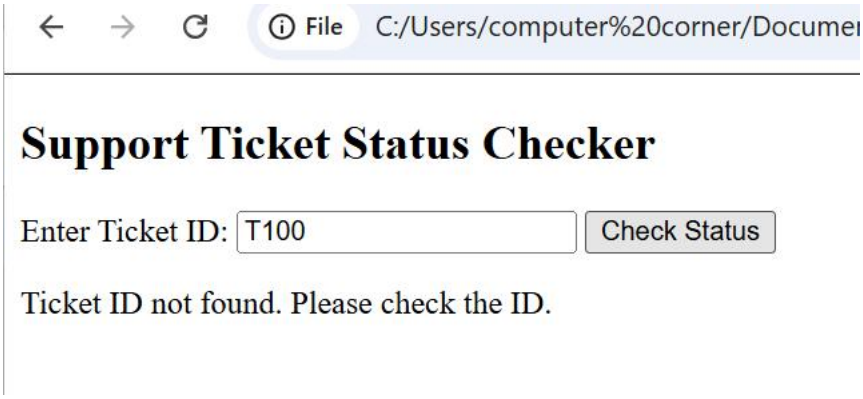
Ticket ID: T102 | Customer: Sara | Status: In Progress



Support Ticket Status Checker

Enter Ticket ID: T103 Check Status

Ticket ID: T103 | Customer: Ahmed | Status: Resolved


File
C:/Users/computer%20corner/Docume
Support Ticket Status Checker
Enter Ticket ID: T100
Check Status
Ticket ID not found. Please check the ID.